

Conversion de la pièce du 3D vers le 2D

Comment fonctionne chaque composant de `world.tscn` après la bascule 2D, comparé à l'ancienne version 3D — pour reprendre le développement et les graphismes en autonomie.

1. Pourquoi ce changement

La pièce (le bureau où tu paries, achètes des objets et lances les courses) était construite en 3D avec des assets d'un pack acheté ("Retro PSX Style Mansion Assets"), une `Camera3D` animée par tween, et des pions/objets en `.glb`. Comme ces modèles n'étaient que des prototypes, on est repassé en 2D pour simplifier la suite du développement : plus de pipeline Blender/GLTF à gérer, plus de caméra à animer en 3D, plus de collisions 3D — juste des `Control`, des `Button` et des couleurs à changer.

La piste de course (`oval_track.gd` / `race_track.tscn`) était **déjà** en 2D avant cette conversion (Path2D, Sprite2D) — elle n'a pas été touchée, à part le recentrage de son point d'origine (voir §7).

2. Vue d'ensemble : la correspondance 3D → 2D

Le principe général : chaque mécanisme 3D avait un équivalent 2D plus simple. Voici la table de correspondance utilisée dans tout le code réécrit.

Rôle	Version 3D (avant)	Version 2D (maintenant)
Nœud racine de la pièce	Node3D + Camera3D	Control (plein écran, ancré)
Détection de clic sur un objet	Area3D + CollisionShape3D + signal input_event	Button + signal pressed – aucune collision à poser
Surbrillance à la sélection	Matériau dupliqué, emission_enabled = true	Changement de <code>modulate</code> (teinte du nœud)
"Zoom" vers un écran (carte, boutique)	Tween sur Transform3D de la Camera3D (position + rotation)	Tween sur <code>modulate:a</code> (fendu) + scale d'un Control
Décor (murs, sol, table...)	Meshes GLTF/FBX importés, éclairés par des SpotLight3D	Un <code>ColorRect</code> de fond (placeholder) – à remplacer par une image
Pions des chevaux	<code>statut.glb</code> instancié, positionné en Vector3	Button positionné en pixels (Vector2)
Objets boutique/bureau	<code>.glb</code> par objet (stimulant, sabotage...), visibilité togglée	Label par objet (même logique de visibilité, juste 2D)

3. Nouvelle arborescence de `world.tscn`

Tout est maintenant un seul arbre de `Control`, sans `CanvasLayer` séparé (plus nécessaire puisqu'il n'y a plus de contenu 3D en dessous à superposer).

```
World (Control, script: gamecamera.gd)
├─ Background (ColorRect)      – fond de la pièce, à remplacer par une image
|   └─ BackgroundLabel         – texte "placeholder", à supprimer
├─ RoomView (Control)         – toute la pièce (voir §5 : mouse_filter)
|   └─ MapDesk (Button)        – "Carte des paris" → zoom vers l'écran de course
|   └─ DeskView (Control)      – vue "bureau" (par défaut)
|       └─ Pions (Control)
```

```

| | | |— Pion1..Pion4 (Button) — un par cheval
| | |— Bureau (Control)
| | | |— Slot1..Slot4 (Control) — objets équipés, un Label par type
| | | |— BureauSupplementaire — effet visuel du mod "bureau+"
| | |— ShopView (Control, caché) — vue boutique (BOUTIQUE / RETOUR)
| | |— Emplacement1..3 (Button) — un Label par type d'objet, un seul visible à la fois
|— Main (instance de main.tscn, caché) — l'écran de course, déjà en 2D
|— BtnBack / PanelMoney / PionBg / PionPopup / BtnShop / BtnBackShop
|— Debug (Node, script: debug.gd) — raccourcis F1/F2/F3, inchangé

```

À RETENIR

Chaque "Slot" / "Emplacement" contient 6 `Label` (un par type d'objet : rumeur, sabotage, stimulant, paris, sabodiscret, deskupgrade), tous invisibles par défaut. Le script affiche celui qui correspond à l'objet actuellement possédé/en vente. C'est exactement la même logique qu'en 3D — seul le type de nœud change.

4. Détail par fichier

SCÈNE **world.tscn**

Remplace entièrement l'ancienne scène 3D. Les positions des boutons sont des coordonnées en pixels fixes (écran 1280×720) — c'est la première chose à retoucher quand tu auras un vrai décor : ouvre la scène dans l'éditeur 2D et fais glisser chaque bouton/emplacement à l'endroit exact où il doit se trouver sur ton image de fond.

Élément	Position actuelle (placeholder)
MapDesk	x: 490–790, y: 260–380 (centre de l'écran)
Pion1–4	y: 560–660, espacés de 140px, à partir de x: 380
Slot1–4 (bureau)	y: 40–120, espacés de 110px, à partir de x: 435
Emplacement1–3 (boutique)	y: 250–470, espacés de 280px, à partir de x: 250

SCRIPT **scripts/gamecamera.gd**

C'est le chef d'orchestre de la pièce, attaché au nœud **World**. Son rôle n'a pas changé conceptuellement : il récupère les nœuds de la scène (**@onready var**), crée les 4 sous-systèmes (**HudDisplay**, **MapCameraController**, **ShopManager**, **PionManager**, **DeskManager**) et gère la popup de sélection partagée.

Ce qui a changé

- Le nœud s'attend maintenant à un **Control** (**extends Control**) au lieu d'un **Node3D**.
- Les chemins **@onready** ont perdu le préfixe **CanvasLayer/** et **Room/** puisqu'il n'y a plus qu'un seul niveau de **Control** (ex : **\$CanvasLayer/BtnShop** → **\$BtnShop**).
- Plus de référence à une **Camera3D** — supprimée entièrement.
- _process()** ne fait plus tourner les pions (c'était un effet visuel 3D, **process_rotation()** a été retiré ; tu peux le recréer en 2D avec une simple rotation de **Button** si tu veux garder cet effet).

```
func _ready():
    hud = HudDisplay.new()
    add_child(hud)
    hud.setup(panel_money, label_cash_world, label_tranche_world)

    map_nav = MapCameraController.new()
    add_child(map_nav)
    map_nav.setup(room_view, main_ui, btn_back, btn_shop) # room_view, pas juste desk_view !
    map_desk.pressed.connect(map_nav.zoom_in)
    ...
```

Au départ, `map_nav.setup()` recevait `desk_view` (juste le bureau). Résultat : en zoomant vers la carte, le bouton "Carte des paris" restait affiché par-dessus l'écran de course, car il fait partie de `RoomView` et non de `DeskView`. La correction : passer `room_view` (toute la pièce) à `MapCameraController`, pour que le zoom cache le bouton en même temps que le reste.

SCRIPT `scripts/world/map_camera.gd` (classe `MapCameraController`)

Gère la transition entre la pièce et l'écran de course. En 3D, "zoomer" voulait dire animer la position et la rotation d'une `Camera3D` vers des valeurs précalculées (`_zoom_cam_pos`, `_zoom_cam_rot`). En 2D, il n'y a plus de caméra : on fait juste un fondu entre deux vues.

Avant (3D)

```
var tween = create_tween()
tween.set_parallel(true)
tween.tween_property(
    _camera, "position",
    _zoom_cam_pos, 1.0
).set_trans(Tween.TRANS_CUBIC)
tween.tween_property(
    _camera, "rotation",
    _zoom_cam_rot, 1.0
).set_trans(Tween.TRANS_CUBIC)
```

Après (2D)

```
var tween_out = create_tween()
tween_out.tween_property(
    _room_view, "modulate:a",
    0.0, 0.25
)
await tween_out.finished
_room_view.visible = false
# ... puis on affiche Main en fondu
```

Aucune valeur magique à régler (position/rotation de caméra) : juste une durée de fondu et un léger effet de `scale` pour donner un peu de "pop" à l'apparition de l'écran de course.

SCRIPT `scripts/world/pion_manager.gd` (classe `PionManager`)

Gère le clic sur un pion (cheval) et son animation de sélection.

Mécanisme	3D (avant)	2D (maintenant)
Détecter le clic	<code>Area3D</code> enfant "ClickArea" + <code>CollisionShape3D</code> , signal <code>input_event(camera, event, position, normal, shape_idx)</code>	Le pion est directement un <code>Button</code> , signal <code>pressed()</code> — pas de nœud enfant nécessaire
Surbrillance	Dupliquer le matériau du mesh, <code>emission_enabled = true</code> , couleur d'émission	<code>pion_node.modulate = Color("#e8d5b0")</code>
Animation "sélectionné"	Tween sur <code>global_position</code> (vers la caméra) + <code>scale</code> (Vector3)	Tween sur <code>scale</code> (Vector2) uniquement — pas besoin de déplacer le bouton

DÉTAIL UTILE

Pour que le zoom du bouton (`scale`) se fasse depuis son centre et non son coin supérieur gauche, chaque bouton reçoit `pivot_offset = pion.size / 2.0` au démarrage. Si tu ajoutes de nouveaux boutons animés en `scale`, pense à

faire pareil.

SCRIPT scripts/world/shop_manager.gd (classe ShopManager)

Même traitement que `PionManager`, mais pour les 3 emplacements de la boutique. La seule différence conceptuelle avec l'ancienne version : "zoomer vers la boutique" ne fait plus tourner une caméra vers un autre coin de la pièce 3D — ça bascule simplement entre `DeskView` et `ShopView` (deux `Control` qui se cachent/s'affichent en fondu). C'est plus simple à comprendre, mais ça veut aussi dire que la boutique n'est **pas** un "endroit" physique dans le décor — c'est un écran à part entière. Si tu veux que la boutique soit un meuble visible dans le décor (comme avant), il faudra repenser `zoom_to_shop()` pour faire un panoramique/zoom de caméra 2D plutôt qu'un simple fondu (voir §8, pistes pour la suite).

SCRIPT scripts/world/desk_manager.gd (classe DeskManager)

C'est le script qui a le **moins** changé : sa logique (afficher le bon objet dans chaque slot du bureau selon les modificateurs actifs) ne dépendait déjà pas de la 3D — juste des changements de type (`Node3D` → `Control` dans les annotations de type). Le comportement est identique bit pour bit.

SCRIPT scenes/oval_track.gd (classe OvalTrack)

Ce script n'a **pas** été converti — il était déjà en 2D (`Node2D`, `Path2D`, `Sprite2D`) avant même qu'on commence à discuter du passage en 2D. La seule modification : le centre de la piste ovale était calé sur `Vector2(400, 300)`, une valeur qui ne correspondait plus au centre réel de l'écran une fois `Main` affiché en plein écran (1280×720) à l'intérieur de `world.tscn`. Recalculé à `Vector2(480, 360)` pour compenser le décalage de `Vector2(160, 0)` appliqué à l'instance `RaceTrack` dans `main.tscn` (480+160=640, soit le centre horizontal exact de l'écran).

5. Pièges Godot 2D à connaître pour la suite

Le piège du `mouse_filter`

C'est le bug le plus sournois qu'on a rencontré pendant les tests, et il te touchera à nouveau si tu ajoutes des zones cliquables. En Godot 4, tout `Control` a par défaut `mouse_filter = STOP` — y compris un simple conteneur invisible sans aucun contenu visuel. Résultat : si tu poses un `Control` plein écran (même vide) par-dessus ou à côté d'un bouton, il peut "avaler" les clics avant qu'ils n'atteignent ce bouton, **sans aucune erreur affichée**.

RÈGLE À APPLIQUER SYSTÉMATIQUEMENT

Pour tout `Control` qui sert uniquement de conteneur de mise en page (pas destiné à être cliqué lui-même — `RoomView`, `DeskView`, `ShopView`, `Pions`, `Bureau` ...), mets explicitement `mouse_filter = 2` (IGNORE) dans l'inspecteur, sous "Contrôle" → "Filtre de souris" → "Ignorer". Seuls les vrais éléments cliquables (`Button`) doivent garder le filtre par défaut.

Ancres et décalages (`anchors` / `offsets`)

Tous les conteneurs de la nouvelle scène (`RoomView`, `DeskView`, `ShopView` ...) utilisent le préréglage "Plein rect" (ancres à 0/0/1/1) pour occuper tout l'écran, quelle que soit sa taille. Les éléments positionnés en dur (boutons, emplacements) utilisent à la place des `offset_left/top/right/bottom` fixes en pixels — pratique pour un prototype à résolution fixe (1280×720 dans `project.godot`), mais ça veut dire que rien ne s'adapte si tu changes la résolution. À garder en tête si tu vises plusieurs tailles d'écran plus tard.

Scale et `pivot_offset`

Par défaut, un `Control` grandit/rétrécit depuis son coin supérieur gauche. Pour que les boutons "poussent" depuis leur centre quand on les sélectionne, il faut fixer `pivot_offset` à la moitié de leur taille (`pion.size / 2.0`) avant d'animer leur `scale`.

6. Remplacer les placeholders par tes graphismes

Tout ce qui est actuellement un `ColorRect`, un `Label` ou un `Button` texte est un espace réservé assumé — exactement dans l'esprit de ce que faisaient les modèles 3D prototype avant. Voici comment les remplacer sans casser la logique existante :

Élément	Aujourd'hui	À faire quand tu as l'art final
Fond de la pièce	<code>Background</code> (<code>ColorRect</code> uni)	Remplacer par un <code>TextureRect</code> avec ton image de décor (stretch mode "Keep Aspect Covered" recommandé)
Pions (chevaux), emplacements boutique	<code>Button</code> avec texte	Garder le <code>Button</code> (il gère déjà le clic et le survol) mais lui assigner une <code>theme_override_styles/normal</code> avec ta texture, ou passer en <code>TextureButton</code> si tu préfères un contrôle plus direct sur les 4 états (normal/survol/pressé/désactivé)
Objets (rumeur, sabotage...)	<code>Label</code> texte	Remplacer par un <code>TextureRect</code> ou <code>Sprite2D</code> avec l'icône de l'objet — le script ne fait que changer leur <code>.visible</code> , donc n'importe quel type de nœud fonctionne tant que le nom du nœud reste identique (<code>rumeur</code> , <code>sabotage</code> , etc.)

IMPORTANT

Les scripts identifient les objets par **nom de nœud** (via `get_node_or_null(boite_name)`), pas par type. Tu peux donc remplacer un `Label` par n'importe quel autre nœud tant que tu gardes exactement le même nom ("rumeur", "sabotage", "stimulant", "paris", "sabodiscret", "deskupgrade") au même endroit dans l'arborescence.

7. Bugs rencontrés pendant les tests (et leçons)

Symptôme	Cause	Correctif
Clic sur "Carte des paris" : aucune réaction, aucune erreur	<code>DeskView / RoomView</code> (conteneurs invisibles plein écran) absorbaient le clic avant le bouton — <code>mouse_filter</code> par défaut = STOP	<code>mouse_filter = 2</code> (IGNORE) sur tous les conteneurs purs
Le correctif semblait ne "rien faire" après renvoi du fichier	Godot avait la scène ouverte en mémoire et l'a re-sauvegardée par-dessus le fichier corrigé	Fermer complètement Godot avant tout renvoi de fichier pendant qu'on itère dessus
Le bouton "Carte des paris" restait visible par-dessus l'écran de course	Le zoom ne cachait que <code>DeskView</code> , pas <code>RoomView</code> entier (le bouton est un frère de <code>DeskView</code> , pas un enfant)	<code>MapCameraController</code> cache maintenant <code>RoomView</code> en entier
Le terrain de course n'était pas centré à l'écran	Le centre codé en dur dans <code>oval_track.gd</code> (400, 300) ne tenait pas compte du décalage de l'instance <code>RaceTrack</code> (160, 0) dans <code>main.tscn</code> , ni du fait que l'écran fait maintenant 1280×720 en plein écran	Centre recalculé à (480, 360)

8. Pistes pour la suite

- **Décor** : dessiner ou trouver une illustration de bureau/pièce en 2D (vue de face ou légèrement plongeante), puis repositionner chaque bouton/emplacement dans l'éditeur pour qu'ils correspondent aux zones visuelles de l'image.
- **Effet visuel des pions** : si tu veux retrouver l'effet de rotation qu'avaient les pions 3D pendant leur sélection, ajoute un tween sur `rotation` dans `PionManager._on_clicked()` — c'est trivial en 2D (une seule valeur d'angle, pas de `Vector3`).
- **Boutique comme lieu physique** : si tu veux que la boutique reste un "endroit" du décor plutôt qu'un écran séparé, remplace le fondu de `zoom_to_shop()` par un vrai déplacement de caméra 2D (`Camera2D` avec tween sur `position / zoom`) qui pointe vers la zone boutique de ton illustration.
- **Nettoyage** : les anciens assets 3D (`assets/Retro PSX Style Mansion Assets...`, `statut.glb`, `assets/objets/*.glb`, `carte.glb / carte.tscn`) ne sont plus référencés par `world.tscn` — tu peux les supprimer du projet dès que tu es sûr de ne plus en avoir besoin (garde une sauvegarde ou fie-toi à l'historique git).